Thai Harassment Detection

OS-Optimized Pipeline

Operating Systems Final Project — 2025

Group Member

StudentID	Name
6788019	Natdanai Chuwong-apichat
6788021	Uchukorn Ek-areejit
6788034	Duarn Kaeokertkhen
6788139	Nathawat Burapolpiman
6788218	Saharut Srithawong

Topic & Objective

Topic

Build a Thai text harass classifier that demonstrates OS optimization - CPU scheduling, memory management, multiprocessing, synchronization, and I/O management.

CPU Scheduling

sched_setaffinity(), nice()
Pin workers to cores

Memory Management

mmap() vs read()
Zero-copy file loading

Multiprocessing

fork() via Pool
Parallel tokenization

Synchronization

Mutex + Semaphore
Bounded buffer

I/O Management

fsync() strategies
Durability vs speed

File Management

Atomic writes
Page cache + stat()

Data Source

Training Dataset

Toxicity Multilingual Binary Classification Dataset

Author: Alexander Salazar (2025)
Platform: HuggingFace

Language: Thai (filtered from 8 languages)
Format: Parquet → filtered and converted to CSV

22,855 samples | 7.04 MB
Toxic: 8,429 (36.9%) | Non-toxic: 14,424 (63.1%)

Test Dataset

ThaiSafetyBench

Authors: Trapoom Ukarapol et al. (2026)
Platform: HuggingFace

Language: Thai
Format: Parquet (semi-structured)

1,889 malicious Thai prompts
Designed for LLM safety evaluation

Covers culturally-specific Thai risks and harmful scenarios
Used to evaluate model recall on harmful content

Keyword Dictionaries

Toxic + Harm-Intent Keyword Dictionaries

Original Source: ThaiToxicityTweetCorpus (TMU NLP)
Platform: GitHub

Language: Thai
Format: TXT → processed into CSV / JSON

Original toxic keywords: 44
Used to collect toxic tweets via Twitter API

Extended to 81 toxic keywords with additional terms
Harm-intent keywords: 70 across 7 categories

New Training Datasets: https://huggingface.co/datasets/malexandersalazar/toxicity-multilingual-binary-classification- dataset/tree/main/data
New Testing Datasets: https://huggingface.co/datasets/typhoon-ai/ThaiSafetyBench
New Toxic word Dictionary Datasets: https://github.com/tmu-nlp/ThaiToxicityTweetCorpus/blob/master/toxic_keywords.txt

Data Format & Amount of Data

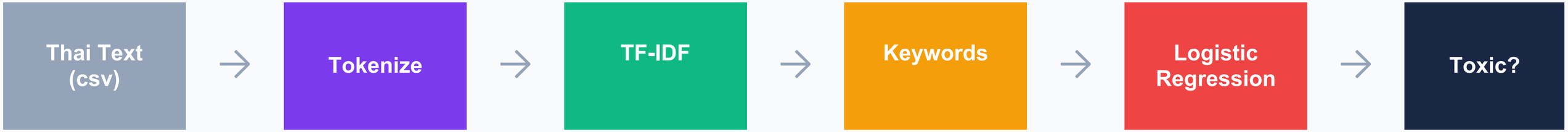
Structured

CSV files with fixed schema
(text + label columns)

thai_toxicity.csv
toxic_keywords.csv
harm_intent_keywords.csv
test.parquet

Dataset	Rows	Columns	Size	Format
Training (thai_toxicity.csv)	**22,855**	2	7.04 MB	CSV
Test (test.parquet)	**1,889**	8	~400 KB	Parquet
Toxic Keywords	81	3	3 KB	CSV
Harm intent Keywords	70	3	3 KB	CSV

Model Architecture



1.Thai Text from CSV — raw Thai sentences, already labeled toxic/non-toxic

2.Tokenize (PyThaiNLP) — Thai has no spaces, so "มึงเป็นขยะ" becomes ["มึง", "เป็น", "ขยะ"]. Uses newmm engine (dictionary matching). This step runs in parallel using fork().

3.TF-IDF — counts how often each word appears in this text vs the whole dataset. Rare-but-frequent words (like "ควาย" in toxic texts) get high scores. Common words (like "เป็น") get low scores. Outputs one big number vector per text.

4.Keyword Matching — checks the text against your 81 toxic words and 70 harm-intent words. Adds extra signals: "does this text contain ควย? ฆ่า? ผิดกฎหมาย?" If yes, flag it.

5.Logistic Regression — takes all the numbers from steps 3+4, multiplies each by a learned weight, sums them up, outputs a probability 0-1. Above 0.5 = toxic.

Training Results

89.5%

Overall Accuracy

18,282 train / 4,571 test

Class	Precision	Recall	F1	Support
Non-toxic	0.95	0.89	0.92	2,943
Toxic	0.82	**0.91**	0.86	1,628
Weighted Avg	**0.9**	**0.9**	**0.9**	4,571

```
Model: tfidf | Accuracy: 89.5%
Trained on: 18,282 samples | Keywords: 81

Demo predictions:

 #  | Label        |  Conf | Text
----+--------------+-------+-----------------------------------------------
 1  | 🟢 Non-toxic  | 92.1% | สวัสดีครับ วันนี้อากาศดีมาก
 2  | 🔴 TOXIC      | 80.8% | ไอ้บ้า มึงไปตายซะ
 3  | 🟢 Non-toxic  | 88.9% | ขอบคุณสำหรับข้อมูลที่เป็นประโยชน์ครับ
 4  | 🔴 TOXIC      | 84.1% | พวกต่างชาติควรออกไปจากประเทศเรา
 5  | 🟢 Non-toxic  | 70.7% | วันนี้ทำอาหารอร่อยมากเลยค่ะ
```

Test Set Evaluation

50.4%

Overall Recall
1,889 harmful prompts

Why is recall only 50.4%?

87% on Hate Speech - the model's core task performs well

Lower on Misinformation / Privacy / Info Hazards
→ These require understanding **intent**, not just vocabulary

Example: "ไม่มีใครอยากคุยกับเธอหรอกนะ" → Emotional harm (implicit harassment)

→ **Needs a transformer model (WangchanBERTa) to understand meaning, not just words**

```
============================================================
 OVERALL RESULTS
============================================================
 Total prompts:    1,889
 Detected toxic:   953  ✓
 Missed (FN):      936  X
 Recall:           50.4%
 Avg toxic prob:   0.474
```

OS: Memory Management — mmap() vs read()

mmap() — Zero Copy

Maps file directly into virtual memory
OS loads pages on demand (page faults)
No kernel→user buffer copy
Best for: large files, random access

read() — Buffered Copy

Copies data: kernel buffer → user buffer
Simpler, immediate availability
Buffer size affects performance
Best for: small files, sequential access

```
──────────────────────────────────────────────────────────────
STEP 1: Data Loading (Memory Management – mmap vs read)
──────────────────────────────────────────────────────────────
  mmap(): 0.1166s | read(): 0.0945s | 22,855 rows | 7204.6 KB
  Difference: 22.2ms

  → Using read()
```

mmap() vs read()

From the benchmark, the read out-performs the mmap().

So we are using read() for managing memory.

OS: Process Management — fork() + Multiprocessing

How it works

1. Parent splits dataset into N chunks

2. Pool calls fork() → N child processes

3. Each child tokenizes its chunk (own PID, own memory)

4. Results collected back in parent via wait()

System calls: fork(), wait(), getpid()

Ran on colab

```
============================================================
BENCHMARK: Process Management — Multiprocessing Workers
  Dataset: 22,855 samples | CPUs: 2 | Parent PID: 5295
============================================================
  1 worker(s): 6.7709s | 22,855 samples | PIDs: {5295}
  2 worker(s): 7.7670s | 22,855 samples | PIDs: {5325, 5326}

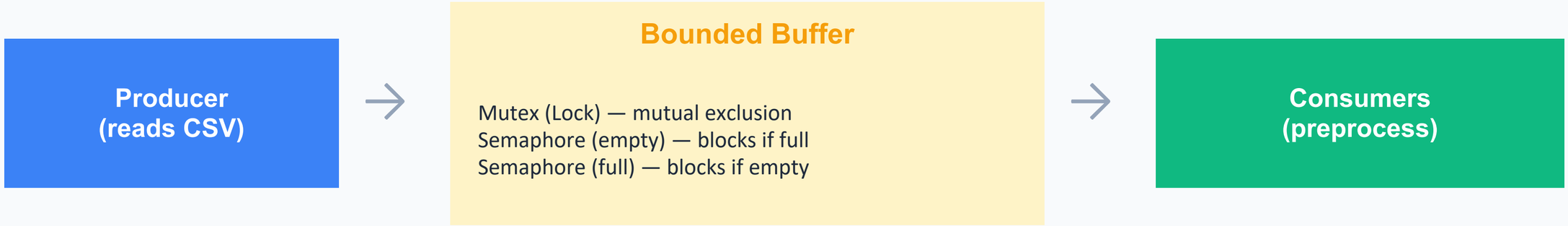
  --- Speedup Analysis ---
  1 workers: 6.7709s | speedup=1.00x | efficiency=100%
  2 workers: 7.7670s | speedup=0.87x | efficiency=44%
============================================================
```

Ran locally on my laptop

```
============================================================
BENCHMARK: Process Management — Multiprocessing Workers
  Dataset: 22,855 samples | CPUs: 10 | Parent PID: 52240
============================================================
  1 worker(s): 2.2354s | 22,855 total (1 chunks of ~22,855) | PIDs: {52240}
  2 worker(s): 1.4630s | 22,855 total (2 chunks of ~11,428) | PIDs: {52250, 52251}
  4 worker(s): 0.9697s | 22,855 total (4 chunks of ~5,714) | PIDs: {52256, 52257, 52254, 52255}
  8 worker(s): 0.9530s | 22,855 total (8 chunks of ~2,857) | PIDs: {52262, 52263, 52264, 52265, 52266, 52267, 52268, 52
269}

  --- Speedup Analysis ---
  1 workers: 2.2354s | speedup=1.00x | efficiency=100%
  2 workers: 1.4630s | speedup=1.53x | efficiency=76%
  4 workers: 0.9697s | speedup=2.31x | efficiency=58%
  8 workers: 0.9530s | speedup=2.35x | efficiency=29%
============================================================
```

OS: Synchronization — Bounded Buffer



Buffer Size	Behavior	Speed
10 (small)	Frequent semaphore waits → blocking	Slower
100 (medium)	Good balance of blocking vs memory	**Balanced**
1000 (large)	Less blocking, more memory usage	Faster

System calls: futex() via Lock, sem_wait()/sem_post() via Semaphore

OS: Synchronization — Bounded Buffer

```
============================================================
BENCHMARK: Synchronization — Bounded Buffer (Producer-Consumer)
============================================================
    Producer: 22,855 items
    Consumer-0: 11,199 items
    Consumer-1: 11,656 items
  Buffer=   10: 0.8338s | 22,855 items | produced=22,857 consumed=22,857

    Producer: 22,855 items
    Consumer-0: 10,843 items
    Consumer-1: 12,012 items
  Buffer=   50: 0.4793s | 22,855 items | produced=22,857 consumed=22,857

    Producer: 22,855 items
    Consumer-0: 10,480 items
    Consumer-1: 12,375 items
  Buffer=  100: 0.4388s | 22,855 items | produced=22,857 consumed=22,857

    Producer: 22,855 items
    Consumer-0: 13,901 items
    Consumer-1: 8,954 items
  Buffer=  500: 0.4209s | 22,855 items | produced=22,857 consumed=22,857

    Producer: 22,855 items
    Consumer-1: 14,337 items
    Consumer-0: 8,518 items
  Buffer= 1000: 0.4659s | 22,855 items | produced=22,857 consumed=22,857

  --- Analysis ---
  Smaller buffer → more semaphore waits (blocking) → slower
  Larger buffer → fewer blocks, but higher memory usage
  The Lock (mutex) serializes access — this is the bottleneck
============================================================
```

OS: I/O Management — fsync() Trade-off

```
============================================================
BENCHMARK: I/O Management — Write Strategies
============================================================
  Buffered (8KB)         : 0.0200s | 5,000 rows | 785.1KB
  Unbuffered (per-row)   : 0.0115s | 5,000 rows
  fsync() per row        : 1.7559s | 500 rows | 285 rows/s
  fsync() per 100 rows   : 0.2036s | 5,000 rows
  fsync() per 1000 rows  : 0.0336s | 5,000 rows

  --- Trade-off Summary ---
  Buffered   → fastest, risk losing data on crash
  fsync/1000 → good balance of speed + durability
  fsync/row  → safest but ~100x slower (disk latency per call)
============================================================
```

What is fsync()?

Forces OS to flush data from page cache to physical disk.

Without fsync: data sits in RAM. Fast but lost on crash.

With fsync: waits for disk confirmation. Safe but slow.

fsync/row = 68x slower than buffered

Strategy	Speed	Durability	Use Case
Buffered	**370K/s**	Low	Temp files, caches
fsync/1000	88K/s	Medium	Log files, batch jobs
fsync/row	**541/s**	Maximum	Financial, critical data

OS: CPU Scheduling & File Management

CPU Scheduling

sched_setaffinity()
Pin workers to specific CPU cores
→ L1/L2 cache stays warm
→ Avoids cold cache misses from migration

nice()
Set process priority (-20 to 19)
→ Prioritize ML inference over data loading

```
============================================================
BENCHMARK: CPU Scheduling — Affinity & Context Switches
  CPUs: 2 | PID: 5266
============================================================

  --- CPU Affinity: pinned vs unpinned ---
  Unpinned (all cores): 0.5404s
  Pinned (1 core each): 0.2653s

  --- Context Switch Overhead ---
   1 processes: 0.1565s
   4 processes: 0.1440s
  16 processes: 0.1644s

  Analysis: CPU pinning keeps L1/L2 cache warm.
  Too many processes → context switch overhead dominates.
============================================================
```

OS: CPU Scheduling & File Management

File Management

Atomic Write Pattern:
1. mkstemp() → create temp file
2. write() → write data
3. fsync() → flush to disk
4. rename() → atomic swap (crash-safe)

Page Cache Effect:
Cold read (disk) → Warm read (cached in RAM)

```
============================================================
BENCHMARK: File Management — Cache, Atomic Write, Metadata
============================================================

  --- Page Cache (cold vs warm read) ---
    Cold: 0.0051s
  Warm 1: 0.0054s
  Warm 2: 0.0053s
  Cache speedup: 0.95x

  --- Atomic vs Direct Write ---
  Direct write: 0.0008s
  Atomic write: 0.0068s (temp→fsync→rename)

  --- stat() syscall speed ---
  10,000 x stat(): 0.0270s (2.7 µs/call)
============================================================
```